State Monad

State · Pure RNG / bank balance simulation · Build with Waqar

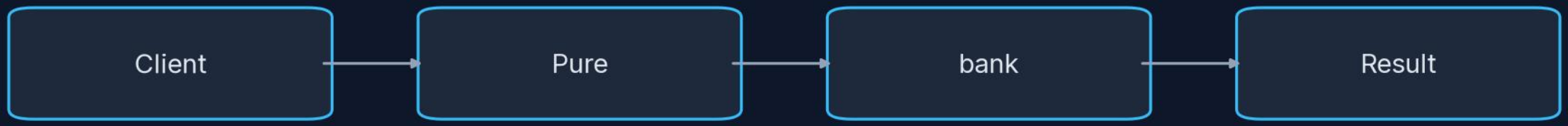
Today's Hook

- Today's force: Pure RNG / bank balance simulation
- We earn State by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: State

The Problem

- Scenario: Pure RNG / bank balance simulation
- Constraints: solve «Pure RNG / bank balance simulation» with State — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Pure RNG / bank balance simulation
- Success looks like: 90 73

Structure



State: Pure RNG / bank balance simulation

Real-World Scenario

- Running example: Pure RNG / bank balance simulation
- Key types: State
- Happy-path stdout cue: 90 73
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```
package com.buildwithwaqar.patterns.functional.state.problem;

import java.util.*;

public final class NaiveRng {
  private int seed; NaiveRng(int s){ seed=s; }
  public int next(){ seed = seed*1103515245+12345; return Math.abs(seed%100); }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Pure RNG / bank balance simulation
- Without State, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.functional.state.problem;

import java.util.*;
import java.util.function.*;

record State<S,A>(Function<S, Map.Entry<A,S>> runF){
  Map.Entry<A,S> run(S s){ return runF.apply(s); }
  static <S,A> State<S,A> pure(A a){ return new State<>(s -> Map.entry(a,s)); }
  <B> State<S,B> map(Function<A,B> f){ return flatMap(a -> State.pure(f.apply(a))); }
  <B> State<S,B> flatMap(Function<A, State<S,B>> f){
    return new State<>(s -> { var r=runF.apply(s); return f.apply(r.getKey()).runF.apply(r.getValue()); });
  }
  static State<Integer,Integer> nextInt(){ return new State<>(s -> { int n=Math.abs((s*1103515245+12345)%100); return Map.entry(n, n==0?1:n); }); }
  static State<Integer,Integer> deposit(int cents){ return new State<>(bal -> Map.entry(bal+cents, bal+cents)); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.state.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== State: pure RNG / bank balance ===");
    System.out.println("--- naive ---"); var n=new NaiveRng(1); System.out.println(n.next()+" "+n.next());
System.out.println("--- state RNG ---");
var rng=State.nextInt().flatMap(a -> State.nextInt().map(b -> List.of(a,b)));
System.out.println(rng.run(1));
System.out.println("--- state bank ---");
System.out.println(State.deposit(50).flatMap(b -> State.deposit(25)).run(0));
  }
}
```

Walkthrough Output

```
=== State: pure RNG / bank balance ===
--- naive ---
90 73
--- state RNG ---
[90, 87]=87
--- state bank ---
75=75
```

Step-by-Step Trace

- Match stdout to stages — look for: 90 73
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 56-state ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
package com.buildwithwaqar.patterns.functional.state.problem;

import java.util.*;
import java.util.function.*;

record State<S,A>(Function<S, Map.Entry<A,S>> runF){
  Map.Entry<A,S> run(S s){ return runF.apply(s); }
  static <S,A> State<S,A> pure(A a){ return new State<>(s -> Map.entry(a,s)); }
  <B> State<S,B> map(Function<A,B> f){ return flatMap(a -> State.pure(f.apply(a))); }
  <B> State<S,B> flatMap(Function<A, State<S,B>> f){
    return new State<>(s -> { var r=runF.apply(s); return f.apply(r.getKey()).runF.apply(r.getValue()); });
  }
  static State<Integer,Integer> nextInt(){ return new State<>(s -> { int n=Math.abs((s*1103515245+12345)%100); return Map.entry(n, n==0?1:n); }); }
  static State<Integer,Integer> deposit(int cents){ return new State<>(bal -> Map.entry(bal+cents, bal+cents)); }
}
```

Trade-offs

- Use State when «Pure RNG / bank balance simulation» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind State in product language.

Related Patterns

- Name the pattern usually confused with State and separate the forces.
- Sketch the roles for «Pure RNG / bank balance simulation» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Pure RNG / bank balance simulation
- Naive pain → State boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.